

Faster than Light: Latency Measurement Errors in Message-Broker Benchmarks

Gustavo Pedro Ricou and Romaric Duvignau

Abstract—A message broker carries messages between distributed services. Latency benchmarks guide the choice of a message broker. They now report sub-millisecond latencies, finer than some of their clocks resolve and shorter than a thread's wait for a core on a busy host. We timed widely used brokers on one clock. Timed from the broker's acknowledgment of receipt, many messages came out negative, as if delivered before being published. The cause is not clock synchronization but scheduling. The thread that records the acknowledgment waits for a core longer than the message takes. Giving it real-time priority at unchanged load removes most negatives. We did a wide audit of the most used tools across the industry: most of them discard such values uncounted or never measure a one-way latency. A positivity filter on a millisecond clock also deletes the fastest messages. We derive the fraction kept, and a test registered in advance confirms it. In practice a sub-millisecond median can describe the clock, not the broker. A void result also looks sound: a sign check rejected most of our runs. We give benchmark authors checks that cost nothing.

Index Terms—Apache Kafka; benchmarking; clock synchronization; latency measurement; measurement errors; message brokers; scheduling; timestamp resolution.

I. INTRODUCTION

OUR first harness timed each message from the moment the producer learned that the broker had accepted it. By its arithmetic, 62,264 of the 738,730 messages we timed arrived before they were published. By the clock, none did. Timed on the same clock from the publish call, the moment the producer hands the message over, not one of them is negative (Fig. 1a). That is the finding we start from. A latency timed from the broker's acknowledgment can come out negative on a single clock, with no disagreement between clocks to blame, and a run in which that happens often cannot report a latency at all. A per-run sign check rejected 1,321 of our 2,266 runs and left nothing of our own first result usable, although that result had passed every conventional check we knew (Section III).

It matters because engineers choose a message broker, the service that carries an application's messages, by its latency benchmarks. These time each message from the *producer* that publishes it to the *consumer* that receives it. Those benchmarks now report latencies below a millisecond [1]. On such a path both the clock and the scheduler are slower than the message. Several of the tools that produce such comparisons time each message in whole milliseconds (Section V-A), and on a busy host the thread that reads a clock can wait milliseconds for

a core (Section IV-C). The numbers still print, and look like any other latency.

The arithmetic had subtracted two timestamps written by two different threads, and a thread must wait for a core before it can read the clock. That wait is the *scheduling delay*, the time a thread that is ready to run spends queued before a core runs it. When the thread that records the acknowledgment waits longer than the message takes to arrive, the difference comes out negative (Fig. 2). Nothing arrived early. One reading was taken late.

Three small laws govern it (Section IV). Broker benchmarks call the time a message takes to reach the consumer its *end-to-end latency*, and the time the producer waits to hear that the broker has it its *publish latency* [2]. The difference is negative exactly when the publish latency exceeds the end-to-end latency. To leading order, its rate is the chance that the timestamping thread is waiting for a core times the chance that this wait exceeds the end-to-end latency. These waits have a peak in the histogram bucket that holds the scheduler's *slice*, the time it lets a thread run at a turn. A slice is a few milliseconds, the scale of the paths now benchmarked. Moving the scheduling while the host's other work stays fixed moves the rate. Running the timestamping threads at *real-time priority*, ahead of every thread at the default *normal priority*, cut it 7–80×.

The industry meets such values and mostly discards them. Seven of the ten measurement tools we read at source dispose of a latency at or below zero without counting it. Of eleven benchmarking tools we ran, ten never meet a negative, because each takes both timestamps in one process, on one clock, so none measures a *one-way latency*, from one process to another. Apache Kafka's own end-to-end tool reports zero at every percentile on a sub-millisecond path [3], and none of the 43 reports we audited states how many samples it kept (Section V).

What discarding costs is clearest in one of these tools, a broker benchmark whose users were told that negative latencies were clock skew between nodes [4]. It discards every latency that is not positive [5], a rule we call a *positivity filter*. For clocks that disagree, the filter is sensible. But its end-to-end clock counts whole milliseconds, and to that clock a message that arrives before the digit changes took no time at all, which is faster than light too. The filter therefore discards every such message, about half of them on the path we measured and nearly all at some publish rates (Section V-E). In 71 of the 75 embedded-mode settings whose summary we captured, with the whole benchmark in one process, it printed a median of exactly 1.0 or 2.0 ms. Those medians were

G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie.

R. Duvignau is with the Department of Computer Science and Engineering, Chalmers University of Technology, Gothenburg, Sweden. E-mail: duvignau@chalmers.se.

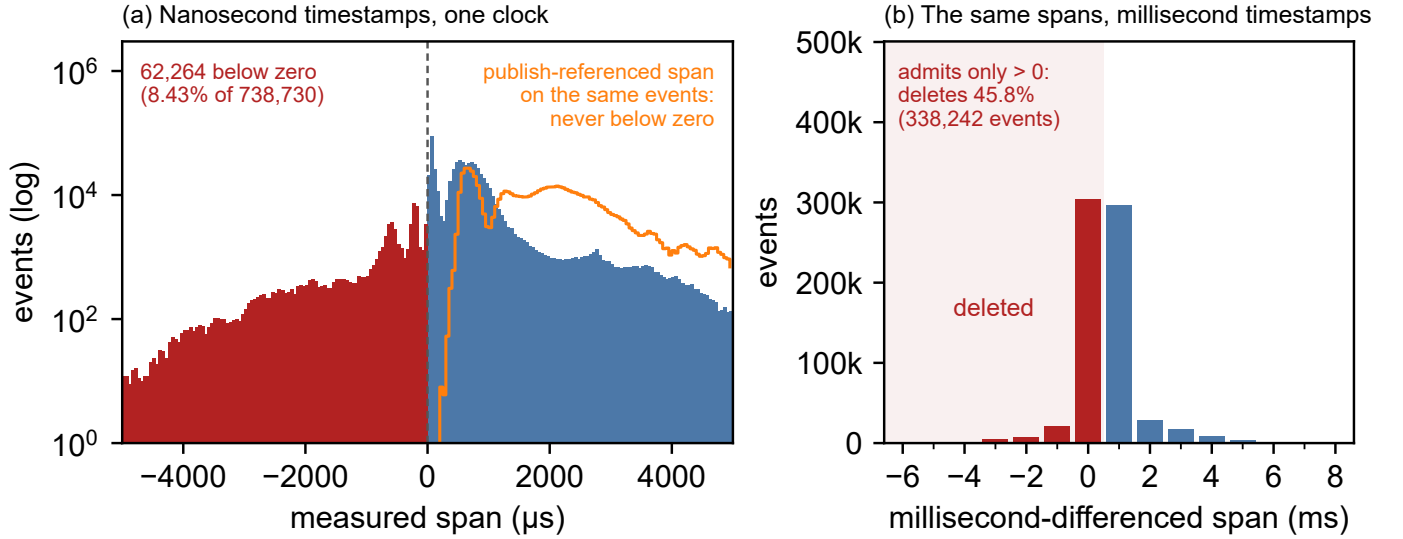


Fig. 1. **Both failures on one population.** The 738,730 messages of our corpus, timed on one clock. (a) Timed from the broker’s acknowledgment, 62,264 of them come out negative (red). Timed from the publish call, none does. (b) The acknowledgment-timed spans of (a) as a millisecond timestamp would record them. A span shorter than a millisecond computes to 0 or 1 ms. A filter that keeps only positive values, a rule several tools apply (Section V), would delete 45.8% of them without a count.

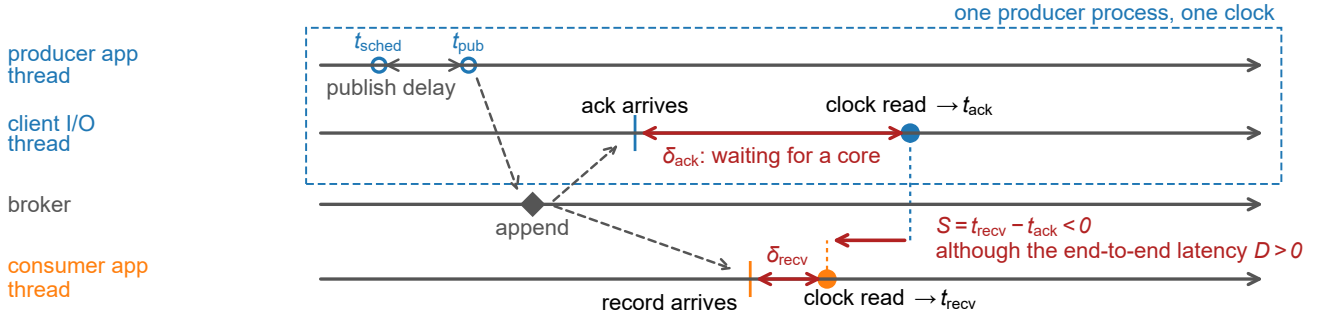


Fig. 2. **Late timestamp, not an early arrival.** How a positive latency is measured as negative, on one clock. The producer’s application thread takes t_{sched} and t_{pub} . Its client library’s I/O thread takes t_{ack} , but only after waiting δ_{ack} for a core. When that wait outlasts the end-to-end latency D , the consumer’s t_{recv} precedes t_{ack} , and $S = t_{\text{recv}} - t_{\text{ack}}$ is negative although D is positive. The path drawn is Kafka’s. Redis timestamps the acknowledgment on the application thread and inverts anyway, because that thread also waits for a core.

computed from as little as 0.36% of the samples taken, and nothing in the output said how few. Applied to the latencies our first harness reported, the filter would remove 45.8% of them uncounted, negatives and zeros alike (Fig. 1b).

Timestamp resolution and scheduling delay are each well understood. What was not known is what they do to the sub-millisecond comparisons that broker benchmarks now report, because the tools that produce those comparisons discard the evidence that would show it. We call the negative latency Failure 1 and the deletion Failure 2.

Neither failure announces itself, and that is the practical problem. On a path faster than a millisecond every sample such a filter keeps reads exactly one millisecond, so the tool prints the same median whatever the broker. It does not print what differs between two brokers, the share of samples kept. A median computed from 0.36% of the samples is typographically identical to one computed from all of them, and more stable from run to run for having its variation deleted. The tool passes the reproducibility a reader checks,

the same median run after run, because it fails the verifiability nobody does.

Where fast and slow messages mix, the deletion inflates instead. At 64 KB, two settings printed medians of 235 and 519 ms, yet most of their messages had arrived within a millisecond. A latency timed from the acknowledgment understates the end-to-end latency it stands for and inflates the gap between two brokers, a property of the clients that a reader would credit to the brokers (Section VI-B). A better-synchronized clock repairs none of this, and the checks that expose it cost nothing (Section VI).

A. Contributions

The paper has three parts and draws their consequences in a fourth. Both failures are established by *manipulation*, moving one variable with the rest held fixed, rather than by observation alone. We call a prediction, campaign or audit *registered* when it was committed in public before the data that test it.

- 1) **The finding** (Section III). A latency timed from an acknowledgment timestamp comes out negative on one clock under both brokers we measured, while none timed from the publish call ever does, and the same negatives appear in Kafka's own Java client and on a second processor family. The check that finds them removes the majority of our runs.
- 2) **The mechanism** (Section IV). Such a latency comes out negative when the thread that writes the acknowledgment timestamp is scheduled late by more than the end-to-end latency. Real-time priority for the timestamping threads cut the rate 7–80× at unchanged utilization, and two placements of the same load at identical utilization differed twofold, so utilization alone does not set the rate. A kernel trace predicts the rate with nothing fitted.
- 3) **The industry** (Section V). With a millisecond clock and a positivity filter, the share of samples a benchmark keeps, its *retention*, is set by the ratio of the end-to-end latency to the timestamp resolution and by where its publish instants fall within each millisecond. A registered manipulation of message size moved a setting across the boundary this predicts. Seven of the ten tools read at source give no count of what they discard (Section V-A). Of eleven tools run under registered predictions, ten start and stop their timing in one process and so cannot measure a one-way latency (Section V-B). A registered audit of 43 public reports found the filter's signature, a median of exactly 1 ms, in eight of the 868 medians they print and a stated retention in none (Section V-G).
- 4) **Practical implications** (Section VI). What a printed latency can and cannot tell its reader, what timing from the acknowledgment costs and how to add it back, and checks that cost nothing: a per-run sign check, and retention reported beside every latency.

II. HOW A BENCHMARK TIMES A MESSAGE

A producer process publishes recorded events, one message each, at a paced rate to a broker, and a consumer process reads them back. Four timestamps bracket a message's path. The first, t_{sched} , is when the event was due to be published, and t_{pub} is taken immediately before the publish call. Then t_{ack} is when the producer learns that the broker accepted the message, and t_{recv} is when the consumer holds the message. A *span* is any difference of two of these timestamps, and the one subtracted is its *origin*. A *negative span* is one below zero. Three spans follow:

$$\begin{aligned} D &\equiv t_{\text{recv}} - t_{\text{pub}}, & A &\equiv t_{\text{ack}} - t_{\text{pub}}, \\ S &\equiv t_{\text{recv}} - t_{\text{ack}} = D - A. \end{aligned} \quad (1)$$

D is the *end-to-end latency*, from the publish call to the consumer holding the message. A is the *publish latency*, from the publish call to the producer observing the broker's acknowledgment. S is what a *harness*, the program that times the messages, reports when it times one from the acknowledgment, as our own first harness did. We call it the *transport proxy*, because it stands in for the broker's transport time without

measuring it. The *event-time latency* E [6] runs from the event falling due to the consumer having handled it, at t_{out} :

$$E = t_{\text{out}} - t_{\text{sched}} = \underbrace{(t_{\text{pub}} - t_{\text{sched}})}_{\text{publish delay}} + A + S + \underbrace{(t_{\text{out}} - t_{\text{recv}})}_{\text{processing-time latency}}. \quad (2)$$

The *publish delay* is how late the publish call ran after the event was due, and the *processing-time latency* is the consumer's own work on the message after t_{recv} .

Write τ for the *timestamp resolution*, the smallest step a timestamp can take, one millisecond in several of the tools of Section V. Write T_{true} for the *true end-to-end latency* of a message, the interval D would report if its timestamps were arbitrarily fine. *Retention* is the fraction of the samples a harness takes that survive into what it reports.

A. Testbeds, clocks and workload

Apart from the registered campaign below, the measured findings of Sections III to V come from four Oracle Cloud VM.Standard.E5.Flex instances, three brokers and one driver, the host that runs the clients. They run Ubuntu 22.04 on kernel 6.8.0-1057-oracle (EEVDF, the Earliest Eligible Virtual Deadline First scheduler) and CPython 3.10 on a private network, with *chrony*-disciplined clocks logged every minute. The producer and the consumer run as separate processes on one host and, unless the text says otherwise, read one clock (`time.time_ns()`), so nothing here depends on clock synchronization.

A workstation running the brokers under containers enters only through the audit of Section III-A. One early stage, unusable because it perturbed the measurement, is excluded from every result here. The campaigns that remain measure utilization with no core pinning. A later registered campaign ran on three pairs of Microsoft Azure machines, two x86 and one Neoverse N2.

The workload is StatsBomb's open football event data [7]: eleven of its 3,315 matches, replayed in order at a chosen publish rate. It stands for small, frequent, ordered messages, and nothing below depends on its content beyond message size and rate (Supplement S1). The achieved publish rate is recorded per run against elapsed wall time, and pacer jitter is 66.3–69.2 μs at p90 over 24 runs.

B. Terms

A *condition* is one point of the experimental design: a broker, a publish rate, a message size, a *background load*, processor work run beside the clients, and a *consumer pattern*, how often the consumer confirms what it has read. A *cell* is a condition together with a workload. Where the workload is fixed, the two coincide. A *run* is one execution of a cell, and a *replicate* is a further run of it. Each setting a manipulation compares is a *configuration*. Utilization ρ is the fraction of processor time busy on the host, as measured over the run.

C. The sign check

A run is rejected if more than 1% of its events carry a negative value in S , the publish delay or the processing-time

TABLE I

NEGATIVES BY SPAN AND BROKER (5,913 RUNS, 738,730 EVENTS, ONE CLOCK). ONLY THE TRANSPORT PROXY S INVERTS, AND IT DOES SO UNDER BOTH BROKERS: KAFKA ON 8.67% OF ITS 367,894 EVENTS, REDIS ON 8.19% OF ITS 370,836, AND THE CAUSAL CHAINS ARE CLEAN UNDER BOTH. THE PUBLISH LATENCY A IS AMONG THEM, ITS SMALLEST RUN MINIMUM 112 μ S CLEAR OF ZERO.

Span	Origin timestamp	Kafka	Redis	All
$t_{\text{recv}} - t_{\text{ack}}$	acknowledgment	31,899	30,365	62,264
$t_{\text{recv}} - t_{\text{pub}}$	publish (chain)	0	0	0
$t_{\text{out}} - t_{\text{pub}}$	publish (chain)	0	0	0
$t_{\text{ack}} - t_{\text{pub}}$	publish (chain)	0	0	0
E , event-time	event time	0	0	0

latency, or if any of their medians is negative. A condition is usable only if all of its runs survive. We fixed the rule before the final campaign, after earlier campaigns had run, and apply it to every run, including those whose results looked reasonable.

The one-percent line is a convention, and it decides nothing we report. Even swept from 0 to 20%, it makes none of our first result's rejected conditions usable (Supplement S2.1). The rule does not assume a negative is impossible, and for S it is not. A run whose origin timestamp fails on more than one event in a hundred cannot report a latency, whatever the cause.

Every proportion below is a count over a stated denominator. Rates carry Wilson score 95% intervals and ratios Katz intervals, Holm-corrected within a named family. The supplement's tables print the corrected values. Every table is generated from the committed data at build time.

III. THE FINDING: NEGATIVE LATENCIES

On one clock, a latency timed from the broker's acknowledgment comes out negative, and no span timed from the publish call does. The sign check of Section II-C finds the negatives in most of our runs.

A. What the sign check finds

The check rejects 862 of 1,382 runs on the workstation testbed (62.4%) and 459 of 884 on the cloud testbed (51.9%). Our first result is among them. The check rejects 109 of the 126 runs behind it, and on that first testbed 8 of 76 conditions survive. Its corpus had survived six corrections, a fairness audit, and a full battery of rank tests, effect sizes and multiplicity correction (Supplement S8.1). None of its six conditions was usable, and nothing in its output said so.

Over a wider population, only one span inverts. Table I separates the spans over every cloud run that produced matched events, 5,913 of them, because a condemned run answers which span inverts as well as a clean one does. The transport proxy S is negative on 62,264 of 738,730 events (8.43%, its *negative-span rate*), and 3,976 runs exceed the threshold on it. The publish-timed spans, which include the publish latency A , and the event-time latency E of Equation 2 are negative on no event at all.

The sign check sees only the worst cases. The publish latency exceeds half the end-to-end latency on 74.5% of events

and a tenth of it on 95.2%, but the check fires only on the 8.4% where it exceeds the whole. The two delays are correlated within a run (median correlation 0.80 over 70 conditions), one scheduler causing both. Treating the two as independent within a run would overpredict the negative-span rate 7.4-fold, the median of the 54 conditions above 0.1%. An end-to-end latency longer than the publish latency is therefore not clean, only quiet. Load-induced inflation that stays positive passes the check by construction.

B. Beyond our clients and our processors

The negatives are not peculiar to our Python clients or to one processor family. A later registered campaign on three further pairs of machines found them in Kafka's own Java client, at a third to a quarter of the Python client's rate, and on Neoverse N2 processors as on x86 (Section VIII). Our timestamp is a CPython call and also waits on the interpreter lock. The Java client, which has none, shows the same failure.

IV. THE MECHANISM: A TIMESTAMP TAKEN LATE

The negatives in our corpus are produced by scheduling. Three small laws carry the argument: an identity that says when the span inverts, a two-state model that says how often, and the scheduler's rule for its slice, which sets the timescale.

A. The transport proxy is not a causal chain

D and E are causal chains, each step preceding the next. S is not. The broker's write of the message precedes both the producer's receipt of the acknowledgment and the consumer's receipt of the message, but neither of them precedes the other. They are two branches of one cause. Since Equation 1 holds per event, whatever the signs,

$$S < 0 \quad \text{if and only if} \quad A > D. \quad (3)$$

Every timestamp is written some time after the moment it marks, because the thread that reads the clock must first be scheduled (Fig. 2). A negative S therefore shows only that the acknowledgment cannot serve as the origin of the end-to-end latency for that event, which is the use an acknowledgment-timed harness puts it to. The same holds wherever the origin timestamp is written by a thread that must first be woken, such as a delivery-report poll or a completion reaper (Supplement S5). The identity is checked on every build. Both sides count 62,264 negatives over our 738,730 events, and the build fails if they part.

A fixed distribution of A overlaps a small D almost entirely and a large one hardly at all. That is why a path whose end-to-end latency runs to tens of seconds passes every run, while the same hardware measuring one millisecond fails wholesale.

B. The cause, by manipulation

Each timestamping thread is either running, in which case its timestamp is late by a narrow jitter, or preempted and waiting for a core. From an idle host to the heaviest load before its processors are fully busy, the rate grows $61\times$ but the jitter's width only $5\times$, so load mostly changes how often

TABLE II

MECHANISM, BY MANIPULATION (2,985 MATCHED EVENTS PER CELL).
 TOP: TIMESTAMPING THREADS MOVED TO SCHED_FIFO WITH THE
 BACKGROUND LOAD UNCHANGED; THE ACHIEVED UTILIZATION ρ OF THE
 TWO CONFIGURATIONS AGREES TO WITHIN 0.0006. BOTTOM: ONE
 UTILIZATION REACHED BY TWO LOAD GEOMETRIES AT $k=6$ LOADED
 CORES, IN AN ORIGINAL CAMPAIGN AND A FULL REPLICATION.

Manipulation	Configuration	ρ	Rate	95% CI	Factor
Priority, 75%	normal	0.7531	0.1320	0.1203–0.1446	39×
	real-time	0.7525	0.0034	0.0018–0.0062	
Priority, 88%	normal	0.8809	0.3049	0.2886–0.3216	54×
	real-time	0.8812	0.0057	0.0036–0.0091	
Geometry, original	concentrated	0.7531	0.0824	0.0731–0.0928	2.07×
	spread	0.7531	0.1709	0.1578–0.1848	
Geometry, repl.	concentrated	0.7531	0.0600	0.0520–0.0691	2.05×
	spread	0.7531	0.1229	0.1117–0.1352	

the thread is waiting for a core. Write p for the probability of the second state, the thread’s *occupancy*. Call the part of that wait still to come the *residual stall*, and write G for its survival function, the chance that it outlasts a given time. Conditioning Equation 3 on the true end-to-end latency gives, to leading order,

$$\Pr[S < 0 \mid T_{\text{true}}] \approx p G(T_{\text{true}}), \quad (4)$$

the occupancy times the chance that a residual stall outlasts the end-to-end latency. The highest rate near saturation, 0.37, is the largest observed rather than a bound.

The core count k is the number of cores the background load occupies. Two configurations that reach the same utilization with the load on different numbers of cores differ in *load geometry*: *concentrated* on few cores or *spread* over many. Load changes p but does not determine it (Table II, bottom), so we write p , not $p(\rho)$, and the model predicts how the rate falls as the path lengthens, not how it rises with load.

Real-time priority on the timestamping threads should cut their occupancy while utilization stays fixed, and the rate falls by factors of 39 and 54 (Table II, top). Across eight matched pairs from 60 to 95% utilization the factor ranges from 7 to 80, every pair with disjoint intervals. A registered prediction that priority would cut the occupancy at least tenfold failed. Scheduler counters, which report means, saw it fall 1.7- to 2.2-fold, so these results rest on the rate and leave the factorization untested (Supplement S3.1 and S3.4). Utilization did not change, and the rate fell in every pair. That refutes any account in which utilization alone sets the rate, including the queueing form we had ourselves adopted.

The residual rates settle near the floor the model names in advance, the rate when the thread is never preempted, $C_0 \approx 0.004$. Over all eight pairs they run from 0.0017 to 0.0144, on both sides of C_0 .

At $k = 6$ loaded cores, two placements of the same load reach $\rho = 0.7531$, identical to four decimals. Their negative-span rates differ by 2.07×, reproduced at 2.05× by a full replication (Table II, bottom). Padding the payload lengthened the transport proxy 77× at a 0.012 utilization spread, and the rate fell 4.1×. That is the direction Equation 3 predicts, and the opposite of what we would see if stress on the host caused the negatives.

A kernel trace on the `sched_wakeup` and `sched_switch` tracepoints [8] shares no probe or data with the application-level rate. It predicts that rate to within a third across three configurations, with nothing fitted. The ratios are 0.78, 1.06, 1.32, no consistent sign between them. Our tracer check, fixed in advance, sets aside the configuration with the middle ratio, and the other two lie on either side of one. The probe is not free. The rate was 0.272 untraced against 0.231 traced ($z = 3.6$ for their ratio), so each prediction is compared with the traced run’s own rate.

In Kafka’s own Java client, a later registered campaign found the same fall with the end-to-end latency and the same effect of real-time priority (Section VIII).

C. Where the timescale comes from

The scheduling delay behind these negatives is not a single heavy tail. Over the 551,956 traced wakeups the counts have three local maxima, or *modes*, which a power law, descending monotonically, cannot have. The last, in the 2–4 ms bucket, holds 10.5% of all wakeups and stands 4.5× its lower neighbor. Above it the counts collapse. They fall 5.0×, 3.4× and 4.7× over the next three octaves and then 357×, each octave a doubling of the wait, whereas a power law would fall by the same factor in every octave (Fig. 3).

Gregg reported the distribution as bimodal in the post introducing `runqlat` [8], and the `timerlat` tracer separates its interrupt and thread components [9]. What we add is the upper mode’s position, consistent with the scheduler’s base slice. The kernel’s documented rule [10] gives the base slice s by scaling a constant by the number of CPUs n , with a cap:

$$s = 0.75 \text{ ms} \times (1 + \lfloor \log_2 \min(n, 8) \rfloor), \quad (5)$$

which gives $4 \times 0.75 \text{ ms} = 3 \text{ ms}$ on the host’s eight CPUs. That slice is derived rather than measured. The kernel’s timer interrupts a busy core once every *tick*, a millisecond on our hosts. With run-to-parity, which lets a running thread finish its slice before a woken thread preempts it, a thread stays within one slice plus one tick of its fair share [11].

A thread woken while another thread holds the core waits out what is left of the running thread’s slice, up to a whole one. A whole slice is 1.02–4.29 times the per-condition end-to-end latencies this corpus measures, 0.70–2.95 ms. The registered campaign timed that wait afterwards, acknowledgment by acknowledgment, and found the rest of a slice, in a reading made after its verdicts (Supplement S6.5). The slice lands on the scale of the end-to-end latency, which is why the check fires on a substantial minority of events.

Read together, Equations 3, 4 and 5 say what the rate depends on. It rises with the occupancy p , which load raises and real-time priority lowers. It falls as the end-to-end latency outgrows the residual stall, whose scale the slice sets. On our kernel, any host with at least as many CPUs as ours has a slice of 3 ms, so every path faster than that is within reach of a single stall. And it owes nothing to how well two clocks agree, because one clock reads both timestamps.

TABLE III

TEN TOOLS, READ AT SOURCE. THE SPAN EACH MEASURES, THE CLOCK IT READS, WHAT HAPPENS TO A VALUE AT OR BELOW ZERO, AND WHETHER THE TOOL COUNTS WHAT IT DISCARDS; ONE ROW PER SPAN WHERE A TOOL MEASURES TWO, AND A DASH WHERE NOTHING IS DISCARDED. THE SOURCE LINE BEHIND EVERY ROW IS IN THE ARTIFACT’S REGISTRY.

Tool	Span measured	Clock	What happens to the value	Counts
Apache Kafka (bundled)	end-to-end	nanosecond	truncate to the resolution	no
Apache Kafka (bundled)	publish	millisecond	truncate to the resolution	no
Apache Pulsar perf	end-to-end	millisecond	filter, ≥ 0	no
NATS CLI	end-to-end	nanosecond	refused by own library	no
OpenMessaging Bench.	end-to-end	millisecond	filter, > 0 , truncate to the resolution	no
OpenMessaging Bench.	publish	nanosecond	—	—
RabbitMQ PerfTest	end-to-end	nanosecond	substitute a value	no
Rezulus	scheduler wakeup	nanosecond	discard	yes
blktrace/btt	block stage	nanosecond	substitute a value	no
emqtt-bench	end-to-end	millisecond	—	—
fio	I/O completion	nanosecond	substitute a value	no
wrk2	end-to-end	microsecond	refused by own library	no

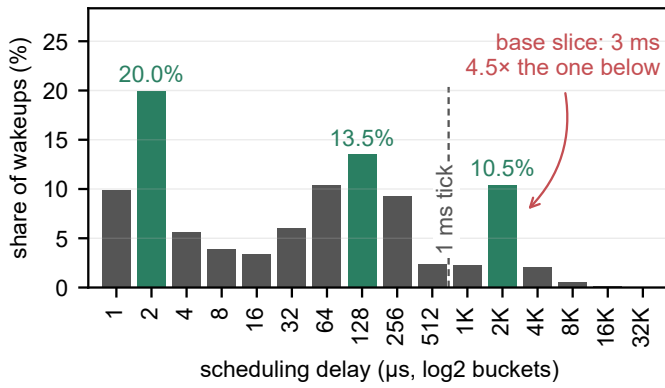


Fig. 3. **Traced scheduling delays are trimodal.** 551,956 wakeups, `bpfttrace log2` buckets, modes shaded. The jitter peaks in the 2–4 μ s bucket, which alone holds 20.0% of all wakeups. A second mode at 128–256 μ s holds 13.5%. The third, at 10.5%, sits in the bucket holding the scheduler’s derived base slice of 3 ms. The count of modes survives a 2 $\times$ widening of the buckets in both phasings.

V. THE INDUSTRY’S REMEDY: DELETION

We assessed how the industry handles a latency at or below zero in four ways, and they agree. The value is deleted, replaced or never measured, and almost never counted. Of the ten measurement tools we read at source, seven dispose of such a value without a count, by filtering it, letting a library refuse it or substituting another (Section V-A). We ran eleven benchmarking tools, four rounds each, under predictions registered before any run. Ten never meet a negative, because each times in one process, on one clock, which also rules out a one-way latency. The one that times across two clocks keeps negatives in its average (Section V-B). We also measured one tool read at source in depth, a broker benchmark whose positivity filter shows what discarding costs on a millisecond clock (Sections V-C to V-F). Of the 43 reports we audited for that filter’s signature, fixed before any was read, none gives its retention (Section V-G).

A. Ten tools, read at source

The disposal is a pattern across the tooling. We read ten tools at source, across five languages and nine vendors, and

classified each with one program, the same for every tool (Table III). Seven dispose of the sample without counting it, in three classes: filtering, refusal by the tool’s own library, and substitution.

Substitution is the worst, because it leaves nothing missing. The tools `fio` and `btt` substitute zero and one nanosecond, and RabbitMQ PerfTest takes the absolute value, so a negative latency enters its distribution as a positive one of the same size [12]. In each, a value never measured enters the distribution. Apache Pulsar’s performance client admits zero [13]. It drops negatives and keeps the zero samples a strict positivity filter deletes, which then pile up visibly at zero. Apache Kafka’s own end-to-end tool keeps every sample and loses it anyway. It fills the array its percentiles come from by integer division, so on a sub-millisecond path every percentile it reports is zero [3].

In November 2025 Kafka’s coordinator metrics adopted the same disposal, clamping negative durations to zero with no counter [14]. Kafka’s record timestamp is itself a millisecond field on the wire, so patching one benchmark does not reach the broker’s own metadata (Supplement S4.7). The failure is not universal. `emqtt-bench`’s histogram keeps every sample, as RFC 7679 asks [15], and one, Rezulus, counts its discards in a metric that names the cause [16]. The NATS command-line tool keeps a negative in the minimum, median and maximum it prints, while its percentile table, recorded into a library that refuses one, leaves it out with nothing to say so [17]. What survives is the narrower claim: the tools that quantize, filter, refuse or substitute do so without counting.

B. Eleven tools, run

Reading a tool’s source shows what it does with a negative value, not whether it can ever produce one. The registered campaign therefore ran eleven benchmarking tools against a broker on a second machine, four rounds each, with predictions fixed from their source before any run (Table IV). A delay added to the path in steps showed that every one of them measures it. Each figure moved 0.92–1.14 ms per millisecond added, for each time the interval it times crosses the delay. Moving back the clock a tool reads, by more than the interval it times, separated the tools that read one clock from those

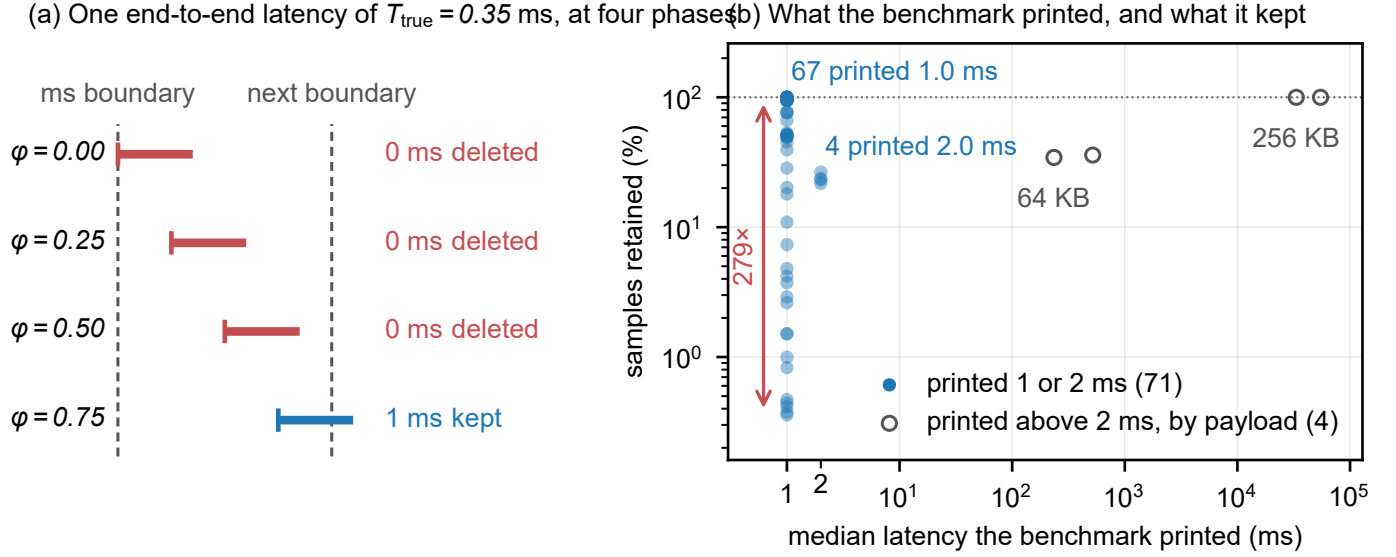


Fig. 4. **Why a sample is deleted, and what the deletion did.** (a) One end-to-end latency of 0.35 ms at four phases of one millisecond. Three finish inside it, compute to 0 ms and are deleted. The fourth crosses a millisecond boundary, one of the *grid* instants at which the clock’s digit changes, and is kept, as exactly 1 ms. (b) Each of the 75 settings with a captured summary, at the median the benchmark printed and its retention. 71 settings print 1.0 or 2.0 ms while retention spans 279 $\times$. Above the grid, nothing is lost only at 256 KB.

that read two. The figures of ten tools did not move, in 70 of their 80 runs and undecided in the rest, because they take both timestamps from one clock, timing a round trip or a message they publish and receive themselves. Such a tool cannot report a negative latency, and for the same reason it cannot measure the one-way latency of the delivery a broker exists to provide.

TABLE IV

ELEVEN TOOLS, RUN. THE REGISTERED CAMPAIGN’S TOOLS BLOCK, FOUR ROUNDS PER TOOL ON ONE PAIR OF MACHINES. INTERVAL: WHAT THE TOOL TIMES, AS ITS INVOCATION WAS READ BEFORE ANY RUN. SLOPE: HOW FAR ITS FIGURE MOVED PER MILLISECOND OF DELAY, DIVIDED BY THE NUMBER OF TIMES THE INTERVAL CROSSES THE DELAY, IN ITS LOWEST AND HIGHEST ROUND. CLOCKS: HOW MANY CLOCKS ITS TWO TIMESTAMPS COME FROM, AS MOVING THE TOOL’S CLOCK BACK BY MORE THAN THE INTERVAL IT TIMES SHOWED. RUNS: THE OFFSET RUNS THAT DECIDED THIS, OF ALL OF THAT TOOL’S. THE REST WERE UNDECIDED.

Tool	Interval	Slope	Clocks	Runs
hey	round trip	1.00–1.04	one	8/8
k6	round trip	1.00–1.02	one	8/8
memtier	round trip	0.96–1.00	one	7/8
Valkey benchmark	round trip	0.98–1.05	one	8/8
Vegeta	round trip	0.99–1.00	one	8/8
wrk2	round trip	0.92–1.04	one	4/8
Kafka producer	to ack.	1.00–1.14	one	8/8
Kafka end-to-end	one process	1.01–1.02	one	6/8
NATS latency	one process	0.99–1.06	one	7/8
RabbitMQ PerfTest	one process	0.97–1.00	one	6/8
librdkafka perf.	two processes	0.96–1.05	two	6/8

The eleventh, librdkafka’s `rdkafka_performance`, timestamps in the producer and subtracts in the consumer. Its figures moved with the clock, and in three of the four rounds it was judged to keep every value, negatives included, in its average, so its figure carries the clocks’ offset along with the path.

C. A positivity filter on a millisecond clock

A filter that admits only positive latencies is one of the ways the tools of Table III dispose of an impossible value, and the one whose cost reaches beyond the negatives. Its origin is documented and its reasoning is sound. In the broker benchmark we measured in depth, the OpenMessaging Benchmark [2], which many public comparisons run, the filter was added in 2018 because end-to-end latency “is calculated based on time difference of `System.currentTimeMillis()` on two different machines” and “can be negative” [5]. Its recorder is an HdrHistogram Recorder, which rejects negative values outright, so filtering for positives is the reasonable local fix, defensive rather than evasive. A tool that times a message faster than light has two choices: announce the discovery, or delete the message. This one deletes it.

The fix is sound locally, and it has two non-local consequences. First, > 0 also deletes every sample that computes to exactly zero. With a millisecond clock, an end-to-end latency shorter than a millisecond computes to zero unless it crosses the *grid*, the instants, one millisecond apart, at which the timestamp changes value. Second, the one observation that would show the measurement had failed is the exact observation the filter removes, and nothing counts it. Over our campaign that silence covers 11,532,492 samples, and Section V-A found the same uncounted disposal in six independent tools.

The coarse clock and the filter sit together on the end-to-end latency, timed in milliseconds, while the same harness times its publish latency in nanoseconds and records every sample. It computes its reported latency distribution from a subset of the samples it collects, does not record how large that subset was, and lets the publish rate decide. We *instrumented* it with counters that record, by sign, every sample its filter discards. Everything measured on it here is in embedded mode, where

the whole benchmark runs in one Java virtual machine (JVM). Its distributed mode failed on every attempt (Section VIII).

D. What the filter did

Of the 75 instrumented settings whose own summary we captured, 71 report a median of 1.0 or 2.0 ms. Those medians were computed from between 0.36 and 100% of the samples the benchmark took (Fig. 4b). Two replicates of one setting, 200 B at 500 msg/s on one host, kept 431 and 120,434 samples, a 279-fold difference. Both reported a median of 1.0 ms, with nothing in the output to distinguish them.

Three uninstrumented runs printed p50, p95, p99 and max all equal to 1.0. Like a stopped clock, the printed median is exactly right whenever the true one happens to be a millisecond. Over all 223 instrumented runs retention falls as low as 0.0044%, about four samples in ninety thousand.

The four settings that print above the grid carry the largest payloads. The two at 256 KB keep every sample. The two at 64 KB keep only 34.4–35.9%. Every sample they delete computes to zero, so most of their messages arrived within a millisecond, and the medians they print, 235 and 519 ms, describe only the slow remainder.

E. The retention law, and a registered confirmation

Retention moves with where the producer's publish instants fall against the grid, their *phase*. Publish latency across these settings takes only two values, 0.3 and 0.4 ms, while retention varies 279-fold. A sample survives when its end-to-end latency crosses a grid instant, which for $T_{\text{true}} < \tau$ happens with probability T_{true}/τ (Fig. 4a):

$$\text{retention} = \min\left(1, \frac{T_{\text{true}}}{\tau}\right) \quad (\text{uniform phases}). \quad (6)$$

All seven configurations whose publish interval is incommensurate with τ , with no ratio of small whole numbers between the two, sit near 50%, with medians from 47.2 to 51.5. That implies $T_{\text{true}} = 0.49$ ms at $\tau = 1$ ms. Where the publish interval is commensurate with τ , retention is confined to a few values, so moving T_{true} moves a configuration between them (Supplement S4.3 tests every configuration). That prediction was registered, and it came true. Replicate spread collapses to 13.6 points at 32 KB and re-opens to 26.7 at 64 KB, a boundary crossed in both directions by one manipulated variable. One registered detail failed. The 32 KB configuration pins at 69.2% retention, not at the predicted 66.67%.

The arithmetic is not ours. Hewlett-Packard's Application Note 162-1 [18] carries the retention identity, the ceiling on resolution gain, the symptom and the cure. What is new is what the harness does with it. The counter keeps both readings, and retention is its estimator. A harness released in 2018 throws away the quantity a 1970 counter measured with.

F. Our own harness, and the sign channel

To show the deletion is not peculiar to one JVM, we reproduced the filter in an independent Python harness sharing

no code with the benchmark. On one clock it recorded zero negative differences across 905,040 samples, as a causal chain requires, and the grid behavior reappeared intact. Counting the discards separately by sign, which we call the *sign channel*, is what makes the filter's cost legible. Across the 214 runs of the benchmark on Kafka, its 10,913,263 discarded samples contain not one negative. The most negative value at any load, size or rate is $0 \mu\text{s}$. Those discards come from the timestamp resolution, not from disagreeing clocks, a distinction an unsigned counter cannot make.

The same channel caught the real thing in the 9 runs of the replication on Redis, which logged 41,403 negatives, every one exactly $-1000 \mu\text{s}$. That is the smallest value a one-millisecond timestamp can express, and it is no coincidence. On Redis the benchmark's span is a difference between two millisecond-floored clocks on two hosts. A sub-millisecond offset between two floored clocks can produce exactly one negative value, $-1000 \mu\text{s}$, and no other. Clock synchronization was clean at every logged minute, so this failure needs no misbehaving clock, only two of them and a floor (Supplement S4.5).

G. Public reports

Failure 2 occurs in practice, but rarely. We registered the audit before reading any report (Supplement S5.1). The signature fixed in advance is an end-to-end median of exactly 1 ms, the smallest value the benchmark can print and the value every retained sample takes on a sub-millisecond path. Of 43 public reports that print the benchmark's end-to-end percentiles, across 1,100 configurations, eight of the 868 medians they print carry it, in two reports: the notebook behind a 2023 journal paper, which shows those medians only in a chart, and the result files of a 2021 fork. None of them also prints a p99 of exactly 1 ms, and no report states how many samples it kept. One reason it is rare is visible in the same reports: 14 of them print medians below a millisecond, which the upstream clock cannot produce, from copies of the benchmark whose maintainers changed how the span is timed.

VI. PRACTICAL IMPLICATIONS

Both failures change numbers that people act on, and neither changes how those numbers look. This section says what a printed latency can and cannot tell its reader, what timing from the acknowledgment costs, what a benchmark author can do about both at no cost, and what is left of the broker comparison.

A. What a printed latency can and cannot say

A broker latency printed below a few milliseconds cannot be taken at face value unless its output says how it was timed and what was kept. For two brokers whose messages all arrive within a millisecond, a tool with a positivity filter on a millisecond clock prints identical output (Section V-E). By Equation 6 the two differ only in retention, T_{true}/τ for uniform publish phases, and retention is never printed. A sub-millisecond ranking read from that output ranks the clock. The error also reaches large messages, where it inflates the

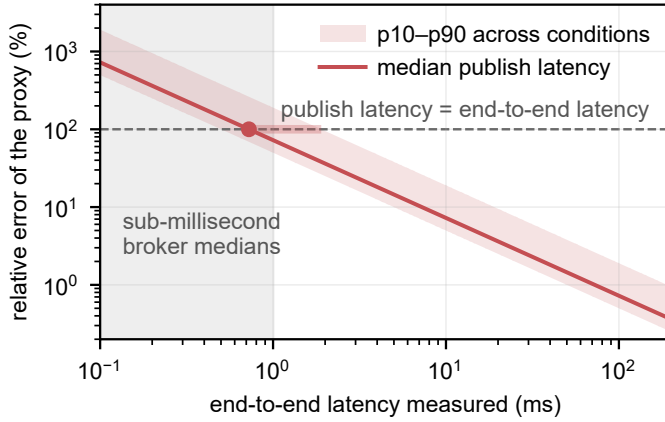


Fig. 5. **Where a path sits on the exposure curve.** The proxy’s error A/D against the end-to-end latency measured, the line at the median publish latency. Across conditions A runs 500–1900 μ s between its tenth and ninetieth percentiles, the band, which puts the error at 10 ms at 5–19% and the crossover at 0.50–1.90 ms. On the dashed line, which the median crosses at 0.72 ms, the publish latency is as long as the end-to-end latency itself, and the shaded decade holds the sub-millisecond medians now reported for brokers. A harness that times from the publish call subtracts no publish latency and has no curve.

median. At 64 KB the printed median described only the slow remainder of a path whose median message took under a millisecond (Section V-D).

No output carries a warning, and none of the 43 reports we audited says what it kept (Section V-G). Most tools avoid the negative instead of reporting it. Ten of the eleven we ran take both timestamps in one process, so their figures are not one-way latencies, and a comparison that sets them beside one-way latencies compares different intervals (Section V-B).

Timed from the acknowledgment, a latency understates the end-to-end latency, the more so the shorter it is. A run of such latencies can be void outright, as our own first result was (Section III-A). Section VI-B sizes the error and repairs it.

The exposure is not confined to benchmarks. Kafka’s coordinator metrics now clamp negative durations to zero without a counter (Section V-A), and any harness that times from a delivery report or a completion reaper starts its clock at a timestamp that may be written late (Section IV-A).

B. The cost of timing from the acknowledgment, and the repair

The publish latency A is a scheduling quantity. It does not grow because a message traveled further, so the relative error of the transport proxy is A/D and falls as $1/D$. At our median publish latency, $A = 725 \mu$ s, the error is 72% at an end-to-end latency of 1 ms, 7% at 10 ms and 0.7% at 100 ms. Below 0.72 ms the median publish latency exceeds the end-to-end latency itself (Fig. 5).

For a harness that reports S , two consequences follow on our corpus. The reported span is 24% of the end-to-end latency it stands for, understating it 4.2 $\times$. That is a median of per-condition ratios over 70 conditions, with an interquartile range of 4.0–7.5 $\times$. And because the two brokers’ clients timestamp differently, the proxy inflates the gap between two systems

TABLE V
CHECKS THAT COST NOTHING, AND WHAT EACH CATCHES.

Check	Catches
Sign-check every run under the load it is quoted at, and report the rejection rate.	Failure 1: an origin timestamp that failed (Section III-A).
Time from the publish call, or record the publish latency and add it back.	Failure 1’s error, A/D (Section VI-B).
Report the retention and the discards counted by sign beside every latency.	Failure 2, and which clock failed (Section V-F).
Read your own percentiles: a median of one clock step, the smallest value printable, means the grid made it.	Failure 2, from output already printed (Section V-D).
Dither the publish instant at random and record the achieved rate.	Retention fixed by the schedule’s phase (Section V-E).

1.7 $\times$ over 24 matched workloads, a property of the clients that a reader would attribute to the brokers.

The publish latency can be added back. Equation 1 holds per event and A is producer-side on one clock. A harness that records its own publish latency therefore recovers the end-to-end latency it failed to measure. Its median error is 3.4% on the 42 conditions the check passes and 12.2% on the 28 it rejects, with upper quartiles of 20% and 19%, two populations that the data cannot tell apart. The repair sits beside the check of Section II-C. The check declines to report the proxy as a latency, and the repair reports the end-to-end latency in its place. Timing from the publish call removes the negative values but not the receipt’s own wait for a core, which inflates every latency where no check can see it. Measured three ways in 51 later runs, that wait added 0.41–0.94 ms to the mean latency on both brokers at 75 and 88% background load, while the median message waited 18–46 μ s (Supplement S3.8 and S3.10).

C. Checks that cost nothing

Both failures are cheap to detect, and Table V lists the checks. SPEC and TPC judge a benchmark on five criteria [19, Sec. 3]. In their terms, the tool of Section I passes the half of *reproducibility* a reader checks, run-to-run consistency at fixed settings, because it fails *verifiability*. A second tester who picks a different publish rate measures a different sample. Better measurement systems exist [20], [21], [22], and the checks apply to whatever they measure.

Every check in the table restores verifiability, and every one has a precedent. Network measurement has reported the share of traces it rejected for nearly thirty years [23], [24]. The One-Way Active Measurement Protocol (OWAMP) records a packet timestamped in the future unaltered rather than discarding it [25]. Randomizing probe times against periodic phenomena is standing advice [26], [27]. What we add is the conjunction, applied where a benchmark reports a latency, with the rejection rate reported as a headline quantity.

One mitigation our measurements support directly costs something. Giving the timestamping threads real-time priority cut the negative-span rate 7–80 $\times$ at unchanged background load. A thread that spins on a core of its own instead of sleeping is *busy-polling*, and never waits to be woken. Busy-polling a dedicated core should buy the same for the price

of that core, though we did not measure it. The floor is not zero, so the check stays, and a real-time or polling timestamping thread changes the system it measures. Where that is unacceptable, report retention instead.

A better-synchronized clock fixes neither failure. Hardware-timestamped Precision Time Protocol [28] would take two hosts from the $67\mu\text{s}$ offset `chrony` reported between ours to under a microsecond. Behind a one-millisecond timestamp the two produce byte-identical records. The filter deletes the same samples, and the retention arithmetic is unchanged. A finer clock does fix the second failure on these paths. It retires Equation 6's deletions and moves the grid to faster paths, but it leaves the filter and every genuine negative in place. The remedies are therefore ordered: record at native resolution, count what is discarded, either the publish instant, and only then does synchronization quality matter.

D. What is left of the broker comparison

Once the failed runs are removed, the broker comparison that motivated the work is small. On the event-time latency E , a causal chain, the two brokers are equivalent against a 40 ms margin (Supplement S7), and on the transport proxy they sit within a millisecond (TOST, two one-sided tests against a 1 ms margin, $p < 0.001$, across three levels). So it is not a purchasing argument. A consumer that confirms each message to the broker and waits for the reply reverses their order, and a co-located testbed hides the effect. Our own measurement treats the two brokers differently. Kafka's client deserializes the payload before t_{recv} and Redis's after it, which is worth $19.5\mu\text{s}$ or 2.4% of Redis's median end-to-end latency. E contains both parses and is free of the asymmetry.

VII. RELATED WORK

That measurement error can dominate a benchmark is established. What prior work does not supply is a check applied to every run, and a count of what the check removes.

A. Broker benchmarks

Broker comparisons appear continuously in peer-reviewed venues and in the gray literature. Apache Kafka and Redis Streams are the pair they most often name, and many of them run the OpenMessaging Benchmark [2]. That experimental setup can dominate a result is established [29], and how to report one is a settled literature [30], [31]. What that literature does not do is check what its tool discards.

Karimov et al. [6] separate event-time from processing-time latency. Van Dongen and Van den Poel [32] use single-broker append timestamps "to ensure correct latency measurements," which is our one-clock rule, reached earlier and by another route. Neither asks what a millisecond timestamp does to a sub-millisecond path, nor which samples the tool keeps. Where a comparison reports a median above the tool's one-millisecond timestamp resolution, the deletion arithmetic of Section V does not bind. Some give no number to place. A 2026 comparison names a percentile for one broker and gives the other two only "sub-millisecond" [1], so its ranking cannot be placed against the resolution at all.

B. Cross-process time

Lamport [33] established that a distributed system has no single physical time, only a causal partial order, and that order decides this paper's central span. Measurement systems defeat clock disagreement [28] by calibrating cross-node counters [20], timestamping in the network interface [21] or the switch dataplane [22], and measuring the timer's own accuracy [34]. Villain et al. [35], profiling FreeBSD against a hardware reference, found that the outgoing socket timestamp "does not respect causality" under every stress pattern, including none. They attributed host latency to interrupt processing, scheduling and context switching. That is the mechanism of Section IV, measured more precisely than we measure it. What it does not give is the consequence for a two-process measurement, that the negative-span rate falls as the end-to-end latency being measured grows. The manipulations that establish it separate scheduling from its rival explanations (Section IV-B).

Paxson [23] treated physically inconsistent transit times as measurement failure and reported the share of traces affected. Lancet [21, Sec. 4.3] reports an inconclusive verdict when a run's samples fail its stability test. Neither reports the share of runs a campaign lost.

Production tracing reached a related answer without a mechanism. Skew adjusters that rewrite a trace's timestamps are called harmful and have defaulted to off since 2020 [36]. Sharma et al. [37] count negative timing spans in distributed inference systems. They hold that "queueing alone cannot produce negative timing spans" and read the negatives as clock skew. For the queue they model, we agree. A second queue decides ours, the run queue the timestamping thread waits in.

C. Resolution and discarded samples

That a timer coarser than the interval it measures invalidates the measurement is textbook [34]. Metrology carries both failures in one uncertainty budget. The NIST stopwatch guide budgets reaction time beside display resolution, and notes that at short intervals the resolution becomes significant against the stopwatch's rated accuracy [38]. That periodic sampling commensurate with a periodic phenomenon sees only part of it is the IP Performance Metrics framework [26], and RFC 3432 makes a randomized start mandatory for periodic streams [27].

Coordinated omission [39] is the mirror image. There the measurement fails to record samples that would have been slow. Here it deletes samples that were fast. Millsap and Holt argue that coarse timing survives in aggregate because the two signed errors cancel [40, Sec. 7.7]. A filter admitting only positive samples removes one sign, and with it the cancellation that defense depends on.

a) *Sources of scheduling delay*: A runnable thread does not always run, even while a core sits idle. Such work-conservation violations are documented for the Completely Fair Scheduler (CFS) [41], which EEVDF has since replaced. Chandrasekar and Kramberger [42] apply a single-parameter queueing law to benchmark bias and find, independently, that the harness's own scheduling inflates the latencies it reports. The load-geometry contrast of Section IV-B refutes only that single-parameter form.

VIII. THREATS AND LIMITATIONS

Attributing the negatives to scheduling is an inference. One clock excludes clock skew, the manipulations of Section IV-B exclude utilization, timer noise and stress, and the kernel trace observes the wait directly. The interpreter lock is bounded twice. The kernel trace, which cannot see the lock, already brackets the observed rate, and Kafka’s Java client shows the same failure. Both testbeds are virtual, so *steal time*, the time one of our virtual processors is ready to run but waits while the hypervisor runs another machine, is bounded, not excluded. Real-time priority, a control inside our virtual machine, still cut the rate.

Disagreeing clocks on different virtual CPUs [43] cannot explain the negatives either. A clock that reads in 90 ns, as ours did, can only be `kvm-clock` or `tsc`, both of which the kernel uses only where the CPUs’ clocks are guaranteed to agree. An offset large enough to produce the deepest negative span would also have driven the end-to-end latency D below zero. Its floor bounds any offset 455-fold below that. Across hosts `chrony` bounds its own error at 4–7 ms per host, so we claim no cross-host bound in general, and a skew threshold such as Sharma et al.’s [37] does not transfer to a loaded machine (Supplement S3.6 and S9.4).

The mechanism requires a timestamping path that can be preempted. Designs that pin a thread to a dedicated core and busy-poll, or that timestamp in hardware, remove the preempted state, and we make no claim about them. The mechanism’s constants are those of one EEVDF-era kernel (6.8), and CFS-era schedulers may shift them.

A registered campaign tested the mechanism’s law on three further pairs of machines. One prediction held on every campaign that tested it, all five on Kafka, that load raises the negative-span rate. On Kafka, real-time priority removed the negatives in every run, and at the one core count whose slice Kafka’s shortest end-to-end latency reaches, the rate’s fall with the end-to-end latency moved with that slice. Its predictions of where the fall sits and how wide it is failed, and Python’s rate exceeded Java’s by more than the registered factor of 1.5 (Supplement S6).

The deletion arithmetic is demonstrated on one benchmark and reproduced in one independent harness. The registered audit of Section V-G found its signature in few public results, so we claim Failure 2 for those and for our own runs only. We read the ten tools of Section V-A at source and did not run them to classify them. The eleven of Section V-B ran on one pair of machines, one processor family and one kernel, at one publish rate. The distributed mode of the benchmark we measured in depth failed on every attempt, so its cross-host exposure rests on source reading and our own two-host harness (Supplement S4.6). Exposure depends on a deployment’s end-to-end latency and publish rate, which is why we recommend reporting retention rather than calling any reported number wrong.

IX. CONCLUSION

A latency benchmark measures the system only while its timestamps are taken faster than the end-to-end latency it

measures. Once the message is the faster of the two, the arithmetic reports it faster than light, and the tool deletes it. Two timestamps written by threads that each wait for a core come out in the wrong order whenever one thread’s wait exceeds the other’s by more than the time between the moments they mark. A timestamp quantized to a millisecond and filtered for positivity deletes every message that arrives before the millisecond digit changes, about half on the path we measured and nearly all at some publish rates. It reports the survivors as exactly one millisecond, by arithmetic the operator never sees.

Neither failure announces itself. A finer clock cures only the second, and neither is expensive to detect. Most of the tools we ran sidestep the first failure without curing it. Timing in one process, on one clock, they never produce a negative, and they never measure a one-way latency either.

The practical rule is short. Before acting on a broker latency below a few milliseconds, ask which timestamp it was timed from, how many samples the tool kept, and how many it discarded, by sign. None of the 43 reports we audited answers the second. Check the spans whose endpoints are timestamped in different places, count what your tool throws away, and report retention beside the number.

ACKNOWLEDGMENT

The authors thank N. Herbst, for the load-generation point that became one of the checks above, for pointing us to the timer study cited in Section VII, and for a reading that reset the abstract.

In accordance with IEEE policy, Anthropic’s Claude assisted with code development and verification, manuscript editing, and literature search. Experimental design, scientific claims, and interpretation are the authors’.

ARTIFACT AVAILABILITY

Code, analysis and manuscript are archived at 10.5281/zenodo.21650031, the measurement dataset at 10.5281/zenodo.21650064; both resolve to the current version, $v4.0.0$, whose files are restricted pending publication. Every number here is recomputed from the committed data by the archived code at build time, and the test suite fails if the text and the data disagree.

REFERENCES

- [1] Index.dev, “NATS vs Redis vs Kafka: message broker comparison,” Practitioner comparison, 2026, <https://www.index.dev/skill-vs-skill/nats-vs-redis-vs-kafka>. Accessed: Sep. 3, 2026.
- [2] OpenMessaging Project, Linux Foundation, “The OpenMessaging benchmark framework,” 2018, available at openmessaging.cloud/docs/benchmarks/. Accessed: Aug. 19, 2026.
- [3] Apache Software Foundation, “Apache Kafka tools — EndToEndLatency.java and ProducerPerformance.java,” Source repository, 2026, <https://github.com/apache/kafka>. Accessed: Aug. 31, 2026.
- [4] OpenMessaging Benchmark contributors, “Can the average publish latency be larger than end-to-end latency? (issue #216),” <https://github.com/openmessaging/benchmark/issues/216>, 2021.
- [5] S. Guo, “Avoid adding negative metric values (pull request #56),” OpenMessaging Benchmark, commit ebb5d6b8, Mar. 2018, <https://github.com/openmessaging/benchmark/pull/56>. Accessed: Aug. 18, 2026.

- [6] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *Proc. IEEE 34th Int. Conf. Data Engineering (ICDE)*, 2018, pp. 1507–1518.
- [7] StatsBomb, "Statsbomb open data," *GitHub Repository*, 2023. [Online]. Available: <https://github.com/statsbomb/open-data>
- [8] B. Gregg, "Linux bcc/BPF run queue (scheduler) latency," <https://www.brendangregg.com/blog/2016-10-08/linux-bcc-runqlat.html>, Oct. 2016, accessed: Sep. 13, 2026.
- [9] D. Bristot de Oliveira, D. Casini, J. Lelli, and T. Cucinotta, "Timerlat: Real-time Linux scheduling latency measurements, tracing, and analysis," *IEEE Trans. Comput.*, vol. 74, no. 8, pp. 2608–2620, 2025.
- [10] Z. Zhou, "sched: Reduce the default slice to avoid tasks getting an extra tick," Linux kernel mailing list, commit 2ae891b82695, 2025.
- [11] V. Guittot, "sched/fair: Manage lag and run to parity with different slices," Linux kernel mailing list, Jul. 2025.
- [12] Broadcom, "RabbitMQ PerfTest — TimestampProvider.java," Source repository, 2026, <https://github.com/rabbitmq/rabbitmq-perf-test>. Accessed: Oct. 2, 2026.
- [13] Apache Pulsar contributors, "PerformanceConsumer.java," Source repository, 2026, <https://github.com/apache/pulsar>. Accessed: Aug. 21, 2026.
- [14] Apache Kafka contributors, "KAFKA-19888: Clamp negative values in coordinator histograms," Issue tracker and pull request #20986, 2025, <https://issues.apache.org/jira/browse/KAFKA-19888>. Accessed: Aug. 21, 2026.
- [15] EMQ Technologies, "emqtt-bench: MQTT benchmark tool — src/emqtt_bench.erl," Source repository, 2026, <https://github.com/emqx/emqtt-bench>. Accessed: Aug. 21, 2026.
- [16] IOP Systems, "Rezulus: systems performance telemetry — scheduler_discarded_samples," Source repository, 2026, <https://github.com/iopsystems/rezulus>. Accessed: Aug. 20, 2026.
- [17] Synadia, "NATS cli — cli/latency_command.go," Source repository, 2026, <https://github.com/nats-io/natscli>. Accessed: Oct. 2, 2026.
- [18] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766.
- [19] J. von Kistowski, J. A. Arnold, K. Huppler, K.-D. Lange, J. L. Henning, and P. Cao, "How to build a benchmark," in *Proc. 6th ACM/SPEC Int. Conf. on Performance Engineering (ICPE)*, 2015, pp. 333–336.
- [20] S. Yang, J. Jeong, B. Scholz, and B. Burgstaller, "Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads," *arXiv preprint arXiv:2205.09325*, 2022.
- [21] M. Kogias, S. Mallon, and E. Bugnion, "Lancet: A self-correcting latency measuring tool," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2019, pp. 881–896.
- [22] R. Kundel, "P4STA: High precision network function benchmarking," in *Accelerating Network Functions Using Reconfigurable Hardware*, ser. Springer Theses. Springer Nature Switzerland, 2024, pp. 93–115.
- [23] V. Paxson, "On calibrating measurements of packet transit times," in *Proc. ACM SIGMETRICS*. ACM, 1998, pp. 11–21.
- [24] —, "Strategies for sound Internet measurement," in *Proc. ACM Internet Measurement Conf. (IMC)*. ACM, 2004, pp. 263–271.
- [25] S. Shalunov, B. Teitelbaum, A. Karp, J. W. Boote, and M. J. Zekauskas, "A one-way active measurement protocol (OWAMP)," Internet Engineering Task Force, RFC 4656, Sep. 2006.
- [26] V. Paxson, G. Almes, J. Mahdavi, and M. Mathis, "Framework for IP performance metrics," Internet Engineering Task Force, RFC 2330, May 1998.
- [27] V. Raisanen, G. Grotefeld, and A. Morton, "Network performance measurement with periodic streams," Internet Engineering Task Force, RFC 3432, Nov. 2002.
- [28] IEEE, "IEEE standard for a precision clock synchronization protocol for networked measurement and control systems," IEEE Std 1588-2019, 2020, revision of IEEE Std 1588-2008.
- [29] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2009, pp. 265–276.
- [30] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. ACM, 2007, pp. 57–76.
- [31] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2015, pp. 73:1–73:12.
- [32] G. Van Dongen and D. Van den Poel, "Evaluation of stream processing frameworks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 8, pp. 1845–1858, 2020.
- [33] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [34] M. Kuperberg, M. Krogmann, and R. H. Reussner, "Metric-based selection of timer methods for accurate measurements," in *Proc. 2nd ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, Mar. 2011, pp. 151–156.
- [35] B. Villain, M. Davis, J. Ridoux, D. Veitch, and N. Normand, "Probing the latencies of software timestamping," in *Proc. IEEE Int. Symp. Precision Clock Synchronization for Measurement, Control and Communication (ISPCS)*, 2012, pp. 1–6.
- [36] Jaeger contributors, "Clock skew adjuster considered harmful (issue #1459)," Issue tracker, 2019, <https://github.com/jaegertracing/jaeger/issues/1459>. Accessed: Aug. 21, 2026.
- [37] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, "Time, causality, and observability failures in distributed AI inference systems," *arXiv preprint arXiv:2604.21361*, Apr. 2026, Open Compute Project, Unified Intelligent Infrastructure workstream.
- [38] J. C. Gust, R. M. Graham, and M. A. Lombardi, "Stopwatch and timer calibrations (2009 edition)," National Institute of Standards and Technology, NIST Special Publication 960-12, 2009.
- [39] G. Tene, "How NOT to measure latency," 2015, talk, Strange Loop / QCon.
- [40] C. Millsap and J. Holt, *Optimizing Oracle Performance*. Sebastopol, CA: O'Reilly, 2003.
- [41] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, "The Linux scheduler: a decade of wasted cores," in *Proc. 11th European Conf. Computer Systems (EuroSys)*. ACM, 2016, pp. 1–16.
- [42] A. Chandrasekar and J. Kramberger, "Identifying and mitigating systemic measurement bias in production LLM inference benchmarks," *arXiv preprint arXiv:2605.24217*, 2026.
- [43] Linux Kernel Documentation, "KVM-specific MSRs: MSR_KVM_SYSTEM_TIME_NEW," <https://docs.kernel.org/virt/kvm/x86/msr.html>, 2026, accessed: Aug. 21, 2026.

Gustavo Pedro Ricou received the B.Sc. degree in physics from the Universidade do Porto, Portugal, and specialized in nuclear fusion science at the Universität Stuttgart, Germany, and the Université de Lorraine, France. He holds an M.Sc. in software design with cloud-native computing from the Technological University of the Shannon, with first-class honors, and is currently pursuing an M.Sc. in statistics and data science at Trinity College Dublin, Ireland. He is a Software Engineer with BoscaSports. His research concerns measurement validity — what a reported number is really evidence of — in computer systems, environmental measurement and sports analytics, using pre-registration and physical-consistency auditing.

Romaric Duvignau received the Ph.D. degree in computer science from the University of Bordeaux, France, in 2015, for work on the maintenance and simulation of dynamic random graphs. He is an Associate Professor and Head of the Network and Systems unit at Chalmers University of Technology, Gothenburg, Sweden. His research concerns distributed and data-driven algorithms, the monitoring of large-scale distributed systems, and stream analytics for cyber-physical systems.